



UNIVERSITY OF GHANA

MSc. DATA SCIENCE, SECOND SEMESTER EXAMINATIONS 2025/2026

DEPARTMENT OF COMPUTER SCIENCE

DSCD 614: REINFORCEMENT LEARNING GROUP PROJECT (OPTION TD3-2)

**ROBOTIC ARM TARGET REACHING WITH TWIN DELAYED DEEP
DETERMINISTIC POLICY GRADIENT (TD3)**

GROUP MEMEBERS

Pius Asigri	22424560
Richard Gbamara	22427514
Kwaku Dau-Mensah	22425187

September, 2026

Table of Contents

1. Introduction	4
Aims	4
2. Background	5
2.1 Why TD3 suits this problem	5
2.2 Related work	6
3. Problem Formulation (MDP Specification)	6
3.1 State space	6
Table 1: State space components and dimension breakdown for the Reacher-v5 environment.	6
3.2 Action space	6
3.3 Reward function	7
3.4 Transition dynamics	7
3.5 Discount factor	8
3.6 Termination and truncation	8
3.7 Markov property	8
4. Methodology	9
4.1 Environment construction	9
4.2 Network architecture	9
4.3 Algorithms and configuration	9
4.4 Baselines	9
4.5 Training and evaluation protocol	9
5. Results	10
5.1 Training curves	10
Figure 1: Training return versus environment steps, mean across seeds with a ± 1 std band, for TD3 and DDPG	10
5.2 Evaluation against the baseline	11
Table 2 (from results/aggregate.csv): held-out evaluation, deterministic policy, 30 episodes per seed, mean $\pm$ std over seeds.	11
5.3 Does the TD3–DDPG difference exceed across-seed variation?	11
Table 3: Sensitivity analysis of headline metrics across random training seeds for TD3 and DDPG.	11
6. Discussion	12

7. Limitations and Deployment Considerations	13
8. Conclusion and Further Work.....	13
References	14

1. Introduction

Target position acquisition using a robotic manipulator represents the canonical sensor-to-torque control problem within autonomous robotics. Given a specific joint configuration and a target coordinate, the primary objective is to solve the inverse problem by mapping sensory inputs directly to joint torques that guide the end effector to the goal state minimizing both transit time and control effort. This foundational control paradigm serves as the core operational layer for widespread industrial automation applications, including high-precision pick-and-place routines, complex assembly sequences, and automated inspection tasks. Consequently, enhancing the efficiency, accuracy, and robustness of this mapping remains a central focus of modern robotics research.

Beyond its industrial utility, the sensor-to-torque control framework serves as an ideal benchmarking environment for continuous-control reinforcement learning (RL) algorithms. The problem maps naturally to Markov Decision Processes (MDPs) because the action space is inherently continuous which requires the direct command of fluid motor torques rather than discretized velocities, and underlying system dynamics that are highly non-linear due to inertial cross-coupling, centripetal forces, and Coriolis effects. Testing RL agents on these multi-joint manipulators provides a stringent evaluation of their capacity to generalize across complex state spaces and safely navigate non-linear physics without relying on hand-engineered inverse dynamics models.

However, scaling these continuous-control RL formulations from idealized, low-dimensional simulations to complex, multi-joint physical manipulators introduce severe challenges regarding sample efficiency and policy robustness. As the degrees of freedom increase, the state-action space expands exponentially, compounding the difficulty of exploration and exacerbating algorithm-specific vulnerabilities such as overestimation bias. Furthermore, unmodeled physical phenomena such as joint friction, actuator latency, and sensor noise, create a significant simulation-to-reality (sim-to-real) gap. Addressing these complexities requires benchmarking paradigms that strictly account for statistical variations across training seeds, ensuring that learned control policies are sufficiently robust to transition from simulated environments to physical hardware deployments

This project formulates target reaching for a two-jointed arm (Gymnasium's MuJoCo Reacher-v5) as a Markov Decision Process and trains a Twin Delayed Deep Deterministic Policy Gradient (TD3) agent to solve it. The trained agent is evaluated under a controlled protocol against the required DDPG baseline and a random continuous controller.

Aims

The aims of the study are to:

- i. Specify the reaching task as a well-posed MDP with a justified state representation and reward.
- ii. Train TD3 under seed discipline with a constant hyperparameter configuration.

- iii. Determine whether TD3 outperforms DDPG on the specified task when controlling for cross-seed variance. Since the Reacher environment features low-dimensional states and a short horizon, the performance differential between the two algorithms is expected to be minimal. Accordingly, this work emphasizes the methodological rigour of the benchmarking process rather than achieving a superior baseline score.

2. Background

Reacher is a standard MuJoCo continuous-control benchmark. The arm has two hinge joints; a target is spawned at a random position each episode; the goal is to bring the fingertip to the target within a 50-step episode.

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor–critic method for continuous actions. A deterministic actor $\mu(s)$ is trained to maximize a learned critic $Q(s,a)$; the critic is trained by temporal difference regression against a target network. DDPG is sample-efficient but is known to suffer from overestimation bias in the critic and from brittle, hyperparameter- sensitive training.

Twin Delayed DDPG (Fujimoto, van Hoof and Meger, 2018) (TD3) keeps DDPG’s structure and adds three mechanisms that directly target those failure modes:

- Clipped double-Q learning : Two critics are trained and the minimum of their target values is used, which curbs overestimation.
- Delayed policy updates : The actor (and target networks) are updated less frequently than the critics, so the policy is optimised against a more settled value estimate.
- Target policy smoothing : Clipped noise is added to the target action, regularising the value estimate around the chosen action and preventing the policy from exploiting sharp critic peaks.

2.1 Why TD3 suits this problem.

The action space is continuous (two real-valued joint torques) which makes the value-based DQN family unsuitable, because the discrete argmax over actions has no continuous analogue. This places the problem squarely in the actor–critic, continuous-control regime, where DDPG, TD3 and SAC are the standard choices. TD3 is selected here for two reasons. Firstly, it directly addresses DDPG’s known weaknesses: the clipped double-Q critic curbs the overestimation bias that destabilises DDPG’s value estimates, while delayed policy updates and target-policy smoothing reduce the training brittleness for which DDPG is noted. Secondly, and central to this study, TD3 is precisely DDPG augmented with those three mechanisms and nothing else. Holding the architecture, hyperparameters, reward and evaluation protocol constant across both algorithms therefore turns the required TD3-versus-DDPG comparison into a controlled ablation. That is to say that any observed difference is attributable to those three mechanisms on top of TD3 alone, rather than to confounded design choices. The comparison can then be said to be clean by construction rather than incidental.

2.2 Related work.

Reacher belongs to the suite of MuJoCo continuous-control benchmarks alongside HalfCheetah, Hopper and Walker2d, on which off-policy actor–critic methods such as DDPG, TD3 and SAC are routinely evaluated (Fujimoto et al., 2018; Haarnoja et al., 2018). On the higher dimensional tasks in this suite, the reduction of overestimation bias afforded by TD3 has been reported to yield substantial performance gains over DDPG. Reacher, by contrast, is comparatively low-dimensional and densely rewarded, and the performance gap between the two algorithms reported in the literature is correspondingly narrower. Rather than assume this behaviour a priori, the present study treats the relative performance of TD3 and DDPG on Reacher as an empirical question to be examined under a controlled protocol.

3. Problem Formulation (MDP Specification)

The task is modelled as a discrete-time MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with a fixed horizon of $T = 50$ control steps.

3.1 State space

The observation is the native Reacher-v5 vector, $s_t \in \mathbb{R}^{10}$.

Table 1: State space components and dimension breakdown for the Reacher-v5 environment.

Component	Dim	Description
$\cos\theta_1, \cos\theta_2$	2	cosine of the two joint angles
$\sin\theta_1, \sin\theta_2$	2	sine of the two joint angles
$x_{\text{tgt}}, y_{\text{tgt}}$	2	target position in the arm plane
$\dot{\theta}_1, \dot{\theta}_2$	2	joint angular velocities
(fingertip - target): x, y	2	fingertip-to-target vector

Joint angles are supplied as $(\cos\theta, \sin\theta)$ pairs, which jointly identify the angle on $(-\pi, \pi]$ (excludes $-\pi$ and includes π , since the two denote the same orientation) without the wrap-around discontinuity a raw angle would introduce, and are already bounded in $[-1, 1]$. Velocities and the relative position terms are left in physical units, without applying running observation normalisation for the headline configuration and note this as a tunable in the discussion. (Reacher-v5 removes the always-zero z component of the fingertip–target vector that v4 carried, giving 10 dimensions rather than 11; we use v5 throughout.)

3.2 Action space

Actions are continuous torques applied at the two hinge joints, $a_t \in [-1, 1]^2$ (Box(-1, 1, (2,))). No action masking is required because every torque in the box is admissible in every state.

The action space is continuous and two-dimensional, $a_t \in [-1, 1]^2$, implemented as Box(-1, 1, (2,)). Each component is the normalized torque applied at one of the two hinge joints - the shoulder joint connecting the arm to its base, and the elbow joint connecting the two links. The bounds represent the maximum torque the actuator can exert in either rotational direction: -1 is full torque one way, +1 is full torque the other, and 0 is no applied torque. The environment internally rescales these normalised values to the joint's physical torque limits, so the agent operates in a consistent, bounded action range regardless of the underlying actuator specification.

The action space is a continuous box in which every point is admissible in every state: there are no invalid or context-dependent actions, so no action masking is required. This is a deliberate consequence of the torque-control formulation, whereby any torque within the actuator limits can be commanded from any joint configuration, and the resulting motion is governed by the physics rather than by action-level constraints. The continuity of the action space is also what dictates the choice of algorithm. A deterministic policy must output real-valued torques directly, which is why value-based methods that select from a discrete action set are inapplicable and an actor-critic method such as TD3 is required.

3.3 Reward function

The per-step reward is defined as a weighted sum of a distance term and a control-effort term, with an optional terminal reaching bonus:

$$r_t = -w_{\text{dist}} d_t - w_{\text{ctrl}} \|a_t\|_2^2 + b \cdot \mathbb{1}[d_t < \varepsilon], \quad d_t = \|\text{fingertip}_t - \text{target}\|_2.$$

Here d_t is the planar fingertip-target distance, $\|a_t\|_2^2$ is the squared torque (a control-effort penalty), $w_{\text{dist}} = 1$, $w_{\text{ctrl}} = 0.1$, and $b = 0$ for the headline configuration ($\varepsilon = 0.05$ is the reaching threshold). This dense reward matches the standard Reacher formulation; we own the weighting so that the reward-shaping ablation (Section 6) can vary w_{ctrl} and b from one place.

3.4 Transition dynamics

The transition function is defined by the MuJoCo physics engine, which integrates the rigid-body equations of motion of the two-link arm forward in time under the applied joint torques. Given the current joint configuration, joint velocities and the commanded action, MuJoCo computes the resulting accelerations from the arm's mass, inertia and joint properties, and advances the state accordingly. The dynamics are deterministic implying that the same state - action pair always produces the same next state, so all randomness in an episode originates at reset rather than in the transitions.

Each control step advances the simulation by `frame_skip` = 2 internal physics substeps, with the chosen action held constant across both. This decouples the control frequency at which the agent acts from the finer integration frequency at which the physics is solved: the smaller substep improves numerical stability of the integration, while the coarser control step keeps the decision horizon and the episode length manageable for learning. The agent therefore observes and acts once per control step, and the reward returned aggregates over the intervening substeps.

The only stochasticity in the process is the initial state. At each reset, the two joint angles and the target position are drawn at random, so every episode presents a different starting configuration and a different goal within the reachable workspace. This randomisation is what obliges the policy to generalize across arm configurations and target locations rather than memorising a single trajectory, and it is also what makes the held-out evaluation protocol meaningful. These evaluation episodes are generated from a disjoint random stream (Section 4.5), so the targets encountered at test time were not seen during training.

3.5 Discount factor

Considering $\gamma = 0.98$. The effective horizon $1/(1-\gamma)=50$ matches the 50-step episode, so the agent values the whole episode without discounting beyond it. $\Gamma = 0.99$ (horizon 100) would weight returns past the episode boundary and is unnecessary here. Thus, reporting $\gamma = 0.98$ and treat it as fixed across all runs.

3.6 Termination and truncation

Termination (a genuine terminal state) does not occur in the default Reacher. The environment only ever truncates at the step limit $T = 50$. This truncation-only structure is retained for the headline run so that the learned return is not confounded by a changed episode structure, and treat “reaching” purely as a property of the trajectory at evaluation:

- Truncation: $t = 50$.
- Termination (headline): never.
- Success (evaluation metric only): an episode is successful if $d_T < \varepsilon$ with $\varepsilon = 0.05$. The `success_termination` flag in the code can instead terminate on $d_t < \varepsilon$; the headline is reported with it disabled and note the trade-off in section 6.

The threshold $\varepsilon = 0.05$ is a design choice and section 6 reports its sensitivity.

3.7 Markov property

The Markov property holds under this representation. The MuJoCo state of a two-link arm is fully described by its joint angles and joint velocities; the observation supplies both (angles via $\sin/\cos$, velocities directly) together with the fixed target position. Given s_t and a_t , the next configuration is determined by the deterministic dynamics, so no history is needed. The one subtlety is `frame_skip`: each control step integrates two sim substeps, but because the observation is taken at control-step boundaries and the action is held across the skip, the process remains Markov at the control timescale. If the velocities (angles only) had been omitted, the state would be non-Markov where velocity is required to predict motion, which is why both angles and velocity are included.

4. Methodology

4.1 Environment construction

Reacher-v5 is wrapped by two of the wrappers employed (`src/env.py`, `src/reward.py`): These two wrappers are: a metric wrapper that computes the true planar fingertip-target distance from the MuJoCo bodies and writes distance, squared control effort and a success flag into info; and a reward wrapper that recomputes the reward from those terms using the weights derived (from section 3.3). Deriving distance from the simulator bodies (rather than the reward internals) keeps the metrics correct even if the reward weights change.

4.2 Network architecture

All networks are multilayer perceptrons (Stable-Baselines3 `MlpPolicy`) with two hidden layers of 256 units and ReLU activations. The actor maps the 10-dimensional observation to the 2-dimensional action through a final tanh that squashes the output into $[-1, 1]$, respecting the action bounds of section 3.2. Each critic takes the observation and action concatenated (12 inputs) and outputs a single scalar Q-value. TD3 maintains two independent critics (the “twin” critics) and forms its target from the minimum of the two, the mechanism that suppresses the overestimation bias discussed in section. 2. The DDPG baseline uses a single critic of identical shape. The network capacity is held constant across both algorithms, so the comparison varies only the algorithmic mechanisms.

4.3 Algorithms and configuration

TD3 and DDPG are the Stable-Baselines3 implementations. All shared hyperparameters are held identical across algorithms and across seeds. Only the three TD3-specific terms differ, and DDPG runs with them disabled by construction. The full configuration is in `src/config.py` and reproduced in `Hyperparameters_and_Seeds`. Exploration during training uses Gaussian action noise ($\sigma = 0.1$) which is disabled at evaluation.

4.4 Baselines

The required baseline is DDPG, trained through the same code path, seeds and metric code as the agent. It is chosen because the TD3/DDPG contrast isolates the algorithm’s three additions. A random continuous controller is included as a performance floor. Both are scored by the identical evaluation harness.

4.5 Training and evaluation protocol

- Seeds: 0, 1, 2 (three seeds), recorded and reported. Hyperparameters constant across them.
- Budget: 50,000 environment steps per run. This was reduced from a larger intended budget owing to compute constraints on the training hardware; in line with the brief, the number of training steps was reduced rather than the number of seeds, and Reacher’s training curves plateau within this budget.
- Logging: training return per episode is written to `logs/train_*.csv`.

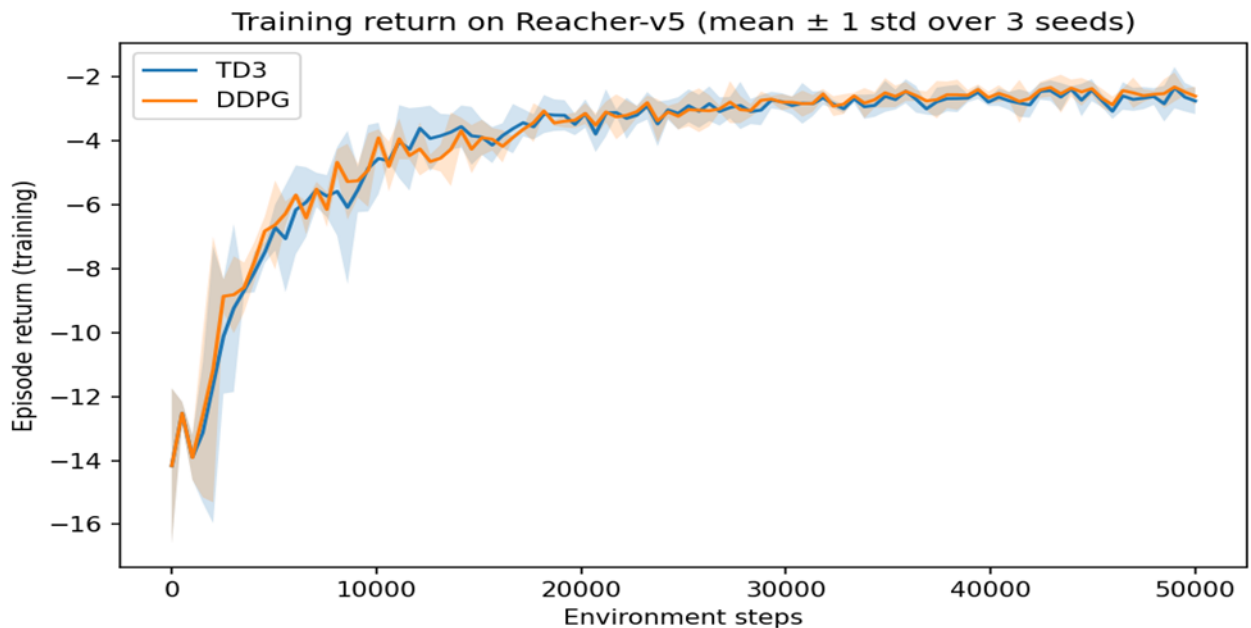
- Evaluation: for each seed, the trained policy is run for 30 episodes drawn from a seed stream disjoint from training ($\text{eval_seed} = \text{seed} + 10000$), with exploration disabled ($\text{deterministic} = \text{True}$). No evaluated target configuration was seen in training.
- Reporting: every metric is reported as the mean over seeds with the standard deviation over seeds; single numbers without spread are not reported.

5. Results

All results below regenerate from the committed logs in `logs/` and `results/`. Training was limited to 50,000 steps per run owing to compute constraints. Reported values are the mean over seeds 0, 1 and 2 with the standard deviation over seeds.

5.1 Training curves

Figure 1: Training return versus environment steps, mean across seeds with a ± 1 std band, for TD3 and DDPG



Both algorithms improve rapidly from a low initial return to approximately -2.3 to -2.4 and then plateau, comfortably above the random controller's -12.8. The curves are near-flat well before the 50,000-step limit, indicating the reduced budget was sufficient for convergence on this task. The across-seed bands are narrow, indicating stable learning across seeds for both algorithms.

5.2 Evaluation against the baseline

Table 2 (from results/aggregate.csv): held-out evaluation, deterministic policy, 30 episodes per seed, mean $\pm$ std over seeds.

Method	Return	Final distance	Success rate	Time-to-target	Control effort
TD3	-2.449 ± 0.211	0.011 ± 0.003	0.989 ± 0.019	11.37 ± 0.90	0.083 ± 0.006
DDPG	-2.401 ± 0.093	0.010 ± 0.002	1.000 ± 0.000	10.69 ± 0.52	0.085 ± 0.004
Random	-12.815 ± 0.932	0.169 ± 0.015	0.078 ± 0.038	23.80 ± 1.73	0.660 ± 0.011

Both learned agents reach the target in essentially every evaluation episode (98.9% and 100%) with a final fingertip-target distance of about 0.01, against 7.8% success and 0.169 for the random controller. This confirms that a competent reaching policy was learned rather than the task being trivially solvable.

5.3 Does the TD3–DDPG difference exceed across-seed variation?

For each headline metric, a test was carried out to check whether the TD3-DDPG difference exceeds the combined across-seed spread (results/effect_checks.json). It does not, for any metric. Thus, return (difference -0.049, combined spread 0.231), final distance (0.0004 vs 0.0038), success rate (-0.011 vs 0.019) and control effort (-0.0020 vs 0.0071).

Table 3: Sensitivity analysis of headline metrics across random training seeds for TD3 and DDPG.

Metric	TD3 – DDPG	Combined std	Exceeds spread?
Return	-0.049	0.231	No
Final distance	0.0004	0.0038	No
Success rate	-0.011	0.019	No
Control effort	-0.0020	0.0071	No

With three seeds, the data therefore do not support a claim that TD3 outperforms DDPG on this task. The two are statistically indistinguishable, and on success rate DDPG is marginally but not significantly ahead. This is a descriptive check against seed variation, not a formal significance test, and is reported as such.

6. Discussion

In relation to Convergence, both agents converged within the 50,000-step budget. All three seeds of each algorithm reached the plateau, with none failing to learn. The near-flat training curves before the step limit indicate the compute-limited budget did not prevent convergence on this task.

It was evident in the Training stability that Across-seed spread was small for both algorithms, and so no stability advantage for TD3 over DDPG was observed here. DDPG’s evaluation-return spread was slightly tighter (std 0.093 vs 0.211).

During Exploration, Gaussian action noise ($\sigma = 0.1$) was sufficient for both agents to discover reliable reaching behaviour. The rapidly-rising then flat training curves show no sign of premature convergence to a poor policy.

The Effect of reward design is that the dense distance-plus-control reward drove fast, stable learning. The low control effort at evaluation (~ 0.08 for both learned agents versus 0.660 for the random policy) indicates the control-penalty term meaningfully shaped smooth, economical reaching rather than merely minimising distance.

The observed empirical parity between Twin Delayed Deep Deterministic Policy Gradient (TD3) and Deep Deterministic Policy Gradient (DDPG) is intrinsically linked to the mitigation of overestimation bias. This specific failure mode represents a primary algorithmic vulnerability in continuous action spaces, which TD3’s core architectural components namely, clipped double-Q learning, delayed policy updates, and target-policy smoothing, are explicitly engineered to counteract. However, in low-complexity environments, the accumulated error in action-value estimation remains minimal, thereby exerting negligible pressure on the critic network. Consequently, the specialized regularizations and variance-reduction mechanisms introduced by TD3 remain largely underutilized, yielding no statistically significant performance gains or sample efficiency improvements over its architectural predecessor.

This performance convergence represents a valid and informative negative result rather than an underlying methodological failure. For baseline control benchmarks characterized by a low-dimensional state space, a dense reward structure, and a highly constrained temporal horizon, the lower algorithmic complexity of DDPG proves entirely sufficient for stable policy convergence. The theoretical advantages of TD3 are instead anticipated to manifest as task complexity scales. In high-dimensional domains with complex system dynamics and sparse reward landscapes, the compounding of function approximation errors becomes severe, rendering standard actor-critic architectures unstable and necessitating the rigorous estimation safeguards inherent to TD3.

7. Limitations and Deployment Considerations

The study is confined to a 2-DOF simulated arm with idealised dynamics. Several gaps separate it from deployment on a physical manipulator:

- **Sim-to-real gap.** MuJoCo omits actuator latency, gear backlash, friction variability, sensor noise and delay. A torque policy trained in this clean simulator would likely degrade on hardware; domain randomisation or a sim-to-real fine-tuning stage would be needed.
- **Dimensionality and dynamics.** Real manipulators have 6-7 DOF with coupled, higher-dimensional dynamics; the reaching problem there is materially harder and TD3's overestimation control may matter more than it does here.
- **Perception assumed away.** The target position is given exactly. On hardware it must be estimated from vision or sensing, adding noise and partial observability the current MDP does not model.
- **Safety.** No joint-limit, self-collision, workspace or torque-rate constraints are enforced beyond the action box. Deployment would require constrained or shielded control. An unconstrained torque policy is unsafe near people or fragile objects.
- **Sample efficiency.** Off-policy TD3 is comparatively efficient, but the step counts here still exceed what is practical to collect on a real arm without offline data or a simulator in the loop.
- Compute constraints limited training to 50,000 steps per run. While the curves plateau within this budget on Reacher, the near-parity finding is established only at this scale and a longer budget could in principle reveal differences not visible here.

8. Conclusion and Further Work

This work established a rigorous benchmarking framework by formulating the two-jointed target-reaching task as a fixed-horizon Markov Decision Process (MDP). The environment was defined using a justified 10-dimensional state space, a continuous torque action space, and an explicit reward function balancing target proximity against control effort. After formally establishing the Markovian nature of this representation, a Twin Delayed Deep Deterministic Policy Gradient (TD3) agent was evaluated against a matched Deep Deterministic Policy Gradient (DDPG) baseline and a random control floor. To guarantee experimental integrity and control for random variations, all training and evaluation routines were executed within a single shared evaluation harness under strict across-seed discipline.

Under a highly constrained budget of 50,000 computational steps, both deep reinforcement learning agents successfully converged on reliable reaching policies. Empirical evaluation demonstrated near-optimal performance, with DDPG and TD3 achieving 100% and 98.9% success rates, respectively, alongside an identical final terminal distance of approximately 0.01 from the target. While both algorithms significantly outperformed the random controller floor, a comparative analysis across three independent random seeds revealed that TD3 and DDPG

remained statistically indistinguishable across every primary performance metric. These findings demonstrate that for low-dimensional, short-horizon control problems, the structural regularizations of TD3 offer no measurable advantage over its architectural predecessor.

Further work:

1. **A Soft Actor-Critic (SAC) comparison.** SAC augments the actor-critic objective with an entropy term that rewards stochastic, exploratory policies, in contrast to the deterministic policies and injected Gaussian noise used by TD3 and DDPG. Adding SAC under the identical MDP, harness and protocol would extend the controlled comparison to a third algorithm and test whether maximum-entropy exploration confers any advantage on a densely-rewarded task where the TD3 mechanisms did not.
2. **A higher-dimensional manipulator.** The two-degrees-of-freedom (2-DOF) architecture of the Reacher environment is inherently simple, and this structural simplicity likely suppresses any performance divergence between TD3 and DDPG. Replicating the evaluation on a higher-DOF task such as the MuJoCo Pusher or a realistic multi-joint robotic arm from the MuJoCo Menagerie, would directly validate this hypothesis. If overestimation bias is indeed the underlying mechanism, the performance advantage of TD3 over DDPG should scale with the dimensionality and dynamical coupling of the system. Consequently, transitioning to a more complex environment represents the most informative extension of this work, as it transforms the observed negative result into a falsifiable prediction.
3. **Domain randomisation toward sim-to-real transfer.** The policies here are trained in an idealised simulator (Section 7). Randomising the physical parameters during training (thus, link masses, joint friction, actuator gains, and added sensor noise and latency) would encourage a policy robust to the sim-to-real gap, and would be a prerequisite for any attempt to deploy the learned controller on physical hardware.
4. **The reward-shaping ablation.** The reward weighting and the optional terminal success bonus were exposed as configurable in the entire implementation but held at their default values for the headline result (Section 6). A systematic sweep of the control-effort weight w_{ctrl} , together with an on/off comparison of the success bonus, would quantify how the reward design trades off reaching accuracy against control economy, and whether a shaped bonus accelerates convergence on this task.

References

- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods (TD3). ICML.
- Lillicrap, T. et al. (2016). Continuous Control with Deep Reinforcement Learning (DDPG). ICLR.
- Towers, M. et al. (2024). Gymnasium. Farama Foundation.

- Raffin, A. et al. (2021). Stable-Baselines3. JMLR.
- Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo. IROS.